

Expt. No. : 15

Date :

REAL-TIME DATA PROCESSING APPLICATION

AIM

To write a python program to implement a GUI application to process a real time stock market data using matplotlib.

SOFTWARE REQUIRED

Python 3.14

THEORY

Introduction to Real-Time Data

Real-Time Data refers to information that is delivered immediately after collection. In various fields, such as finance, telecommunications, and healthcare, real-time data processing is crucial for making timely decisions. Real-time systems must be capable of handling continuous streams of data, updating and presenting it without significant delays.

Characteristics of Real-Time Systems

- **Immediate Processing:** Data is processed and presented as soon as it is received.
- **High Availability:** Systems must be operational at all times to capture incoming data.
- **Responsiveness:** The system must respond to data input quickly, often within milliseconds.

GUI Development with Tkinter

Tkinter is the standard GUI toolkit for Python. It provides a fast and easy way to create user interfaces. With Tkinter, you can create windows, buttons, menus, and various other components.

Key Components of Tkinter

- **Widgets:** The building blocks of Tkinter applications (e.g., buttons, labels, frames).
- **Geometry Management:** Tkinter uses geometry managers like pack, grid, and place to control widget layout.
- **Event Handling:** Tkinter applications are event-driven; they respond to user actions such as button clicks and keyboard inputs.

Data Visualization with Matplotlib

Matplotlib is a versatile plotting library in Python that enables users to create high-quality graphs and visualizations. It is particularly well-suited for scientific and analytical applications.

Features of Matplotlib

- Variety of Plots: Supports line plots, scatter plots, bar charts, histograms, and more.
- Customizable: Users can customize every aspect of the plots, including colors, labels, titles, and legends.
- Interactivity: Matplotlib can create interactive plots that respond to user input in real time.

Interactive Mode

By enabling interactive mode (`plt.ion()`), Matplotlib allows for dynamic updates to plots, making it suitable for applications that require live data visualization. This is particularly important in our application, where the stock price needs to be updated every second.

Random Walk Model in Finance

The Random Walk Model is a statistical model used to represent fluctuating stock prices. This model suggests that price changes are independent of past movements, making stock prices unpredictable over short time intervals.

Key Concepts

- Stock Price Fluctuation: Prices change based on market conditions, news, and investor behavior.
- Random Walk Hypothesis: The hypothesis posits that stock prices follow a stochastic process, where future price movements are random and cannot be predicted based solely on historical data.

Threading in Python

Threading is a technique that allows multiple threads (smaller units of a process) to run concurrently. This is particularly useful in GUI applications to keep the interface responsive while performing long-running tasks, such as data simulation.

Benefits of Threading

- Responsiveness: GUI remains interactive while background processes run.
- Concurrency: Enables the execution of multiple operations at once, such as data collection and user input handling.

Application Workflow

The application workflow can be summarized as follows:

- User Interaction: The user clicks the "Start Simulation" button in the GUI.
- Data Simulation: A separate thread is initiated to simulate stock price changes.
- Data Visualization: As prices are generated, they are plotted in real time using Matplotlib.
- User Feedback: The GUI remains responsive, allowing users to cancel or modify the simulation if needed.

PROBLEM

1. Build a Python application to process real-time heart-rate data and identify readings that fall outside a specified range.
2. Build a Python application to process real-time current measurements in a transmission or distribution feeder and generate an overload warning when the current exceeds the rated value.
3. You are a real estate broker in ancient Knossos. You have m unsold houses, and each house j has an area, x_j , and a minimum price, y_j . You also have n clients, and each client i wants a house with an area greater than a_i and a price less than or equal to p_i . Each client can buy at most one house, and each house can have at most one owner. What is the maximum number of houses you can sell?

ALGORITHM

STEP 1: Start the program.

STEP 2: Import necessary libraries.

STEP 3: Define the RealTimeStockApp class.

STEP 4: Initialize GUI components (window, button).

STEP 5: Define the get_stock_price method for price simulation.

STEP 6: Define the simulate_data method for generating and updating data.

STEP 7: Define the update_plot method for visualizing data.

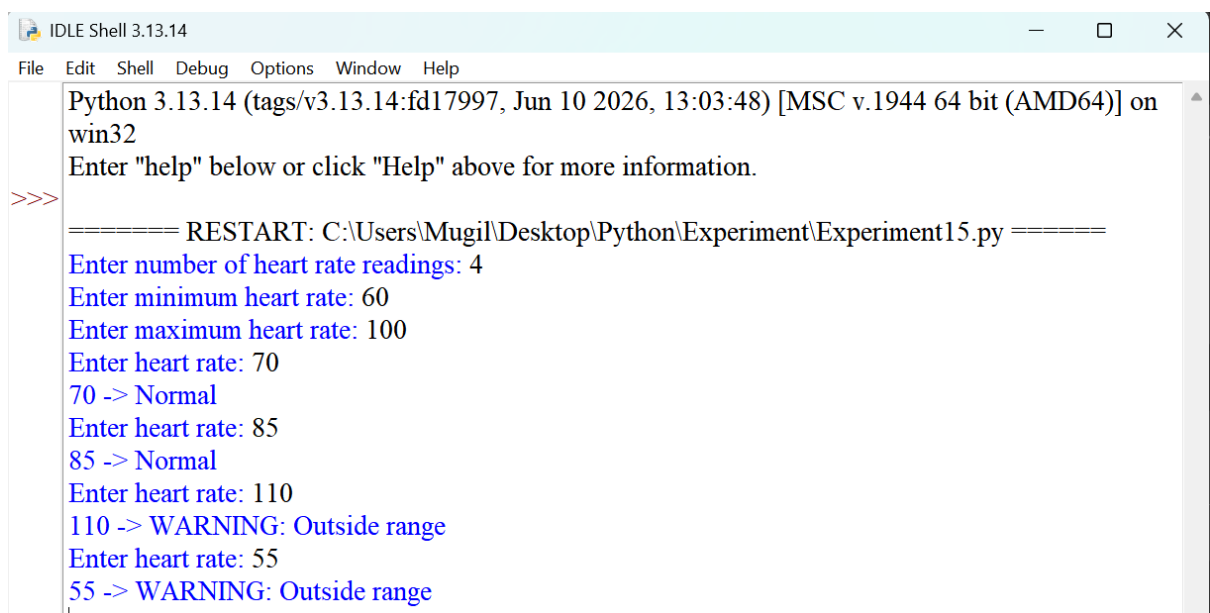
STEP 8: Define the start_simulation method to handle threading.

STEP 9: Create the main program execution block.

STEP 10: Stop the program

PROGRAM CODING

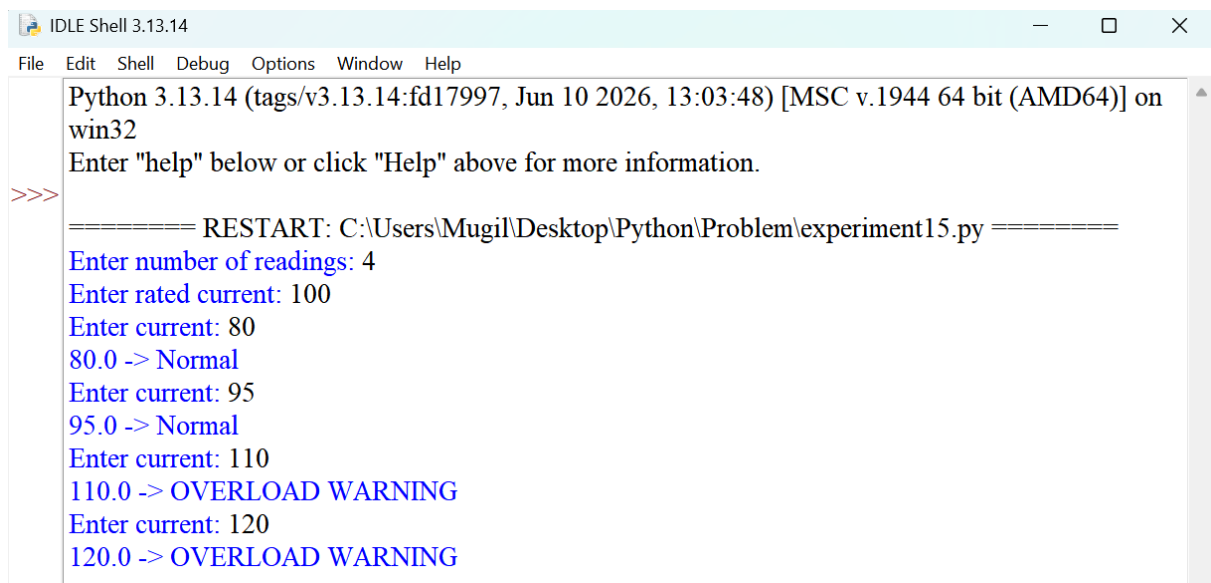
```
n = int(input("Enter number of heart rate readings: "))
low = int(input("Enter minimum heart rate: "))
high = int(input("Enter maximum heart rate: "))
for i in range(n):
    rate = int(input("Enter heart rate: "))
    if rate < low or rate > high:
        print(rate, "-> WARNING: Outside range")
    else:
        print(rate, "-> Normal")
```

OUTCOMES OF PROGRAM CODING

```
IDLE Shell 3.13.14
File Edit Shell Debug Options Window Help
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:\Users\Mugil\Desktop\Python\Experiment\Experiment15.py =====
Enter number of heart rate readings: 4
Enter minimum heart rate: 60
Enter maximum heart rate: 100
Enter heart rate: 70
70 -> Normal
Enter heart rate: 85
85 -> Normal
Enter heart rate: 110
110 -> WARNING: Outside range
Enter heart rate: 55
55 -> WARNING: Outside range
```

PROBLEM CODING

```
n = int(input("Enter number of readings: "))
rated = float(input("Enter rated current: "))
for i in range(n):
    current = float(input("Enter current: "))
    if current > rated:
        print(current, "-> OVERLOAD WARNING")
    else:
        print(current, "-> Normal")
```

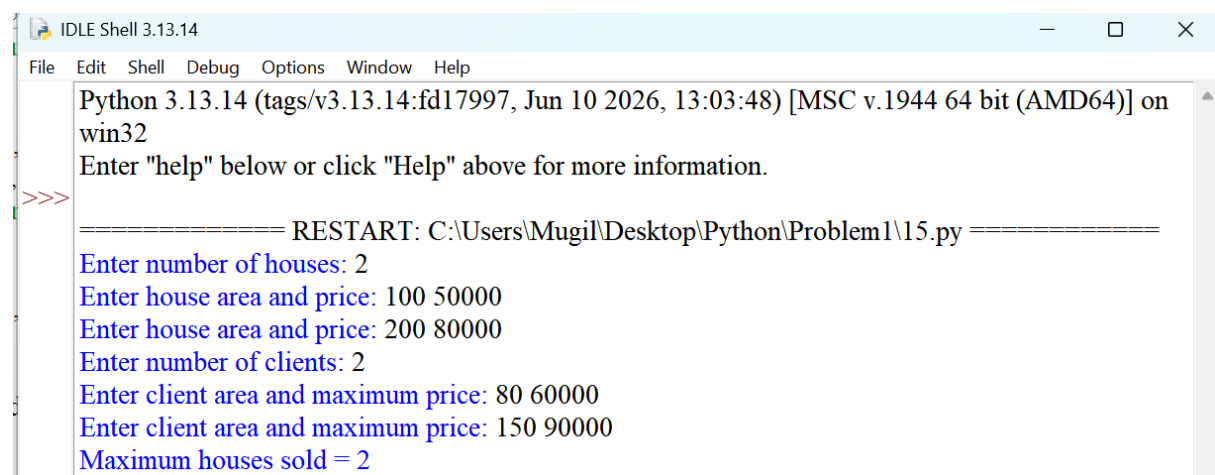
OUTCOMES OF PROGRAM CODINGA screenshot of the IDLE Shell 3.13.14 window. The title bar reads 'IDLE Shell 3.13.14'. The menu bar includes 'File', 'Edit', 'Shell', 'Debug', 'Options', 'Window', and 'Help'. The main text area shows the following output: 'Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32', 'Enter "help" below or click "Help" above for more information.', and a separator line '===== RESTART: C:\Users\Mugil\Desktop\Python\Problem\experiment15.py ====='. Below the separator, the program's execution is shown with prompts and user input: 'Enter number of readings: 4', 'Enter rated current: 100', 'Enter current: 80', '80.0 -> Normal', 'Enter current: 95', '95.0 -> Normal', 'Enter current: 110', '110.0 -> OVERLOAD WARNING', 'Enter current: 120', and '120.0 -> OVERLOAD WARNING'. The prompts are in blue text, and the user input is in black text. The window has standard minimize, maximize, and close buttons in the top right corner.

PROBLEM CODING

```

m = int(input("Enter number of houses: "))
houses = []
for i in range(m):
    area, price = map(int, input("Enter house area and price: ").split())
    houses.append((area, price))
n = int(input("Enter number of clients: "))
clients = []
for i in range(n):
    area, price = map(int, input("Enter client area and maximum price: ").split())
    clients.append((area, price))
used = [False] * m
def match(client, visited):
    a, p = client
    for i in range(m):
        house_area, house_price = houses[i]
        if not visited[i] and not used[i]:
            if house_area > a and house_price <= p:
                visited[i] = True
                if not used[i]:
                    used[i] = True
                return True
    return False
count = 0
for client in clients:
    visited = [False] * m
    if match(client, visited):
        count += 1
print("Maximum houses sold =", count)

```

OUTCOMES OF PROGRAM CODING


```

IDLE Shell 3.13.14
File Edit Shell Debug Options Window Help
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:\Users\Mugil\Desktop\Python\Problem1\15.py =====
Enter number of houses: 2
Enter house area and price: 100 50000
Enter house area and price: 200 80000
Enter number of clients: 2
Enter client area and maximum price: 80 60000
Enter client area and maximum price: 150 90000
Maximum houses sold = 2

```

VIVA QUESTION**1. What is the main goal of your GUI application for processing real-time data?**

The main goal of the GUI application is to provide users with a visual interface to monitor and analyze real-time data, such as energy consumption, sensor readings, or other dynamic metrics. This enables users to make informed decisions based on current trends and alerts.

2. Which programming language and libraries did you use to develop the application, and why?

The application was developed using Python, utilizing the Tkinter library for creating the GUI and Matplotlib for real-time data visualization. Python is chosen for its simplicity and readability, while Tkinter provides a straightforward way to create interactive interfaces, and Matplotlib is excellent for plotting dynamic data.

3. How does the application handle real-time data updates?

The application handles real-time data updates by using a separate thread for data processing. This allows the GUI to remain responsive while continuously fetching and processing data in the background. The data is updated at regular intervals, and the GUI components, including plots and labels, are refreshed to reflect the latest information.

4. Explain how you ensure that the GUI remains responsive during data processing.

To ensure that the GUI remains responsive, the application utilizes Python's threading module. By running the data processing logic in a separate thread, the main thread, which manages the GUI, is free to handle user interactions, updates, and rendering without freezing or lagging.

5. What kind of real-time data can your application process, and how is this data simulated or collected?

The application can process various types of real-time data, such as sensor readings, energy consumption, or stock prices. This data can be simulated using random number generation for demonstration purposes or collected from actual sensors or APIs in a production environment. For instance, in the energy monitoring application, random fluctuations are used to simulate energy consumption.

6. How do you visualize the real-time data in the application?

Real-time data is visualized using Matplotlib. The application maintains lists for timestamps and data values, which are updated continuously. The Matplotlib plot is cleared and redrawn in each update cycle, displaying the latest data and any relevant trends or thresholds to enhance user understanding.

RESULT:

Thus, the application was successfully implemented to process real-time data by continuously receiving, processing, and analysing incoming data values. The processed data was displayed appropriately, and the application was executed and verified successfully.

INFERENCE:

The experiment demonstrates that real-time data processing enables continuous monitoring and immediate analysis of changing data. It is useful for identifying trends, detecting abnormal conditions, and supporting timely decision-making in practical engineering and monitoring applications.

Assessment	Max. Marks	Marks Obtained
Conduction of Practical	15	
Report	5	
Viva	5	
Total Marks	25	
Signature of Lab In-Charge		